PPHA 30538 Section 2
Hani Chung (GitHub: HaniC234)
Cassie Liu (GitHub: ycliu33)

Environmental Exposure and Socioeconomic Vulnerability in Chicago

Research Question

Environmental justice is commonly understood as the intersection between environmental exposure and social vulnerability. However, these two dimensions are often summarized using a single metric, which can obscure how environmental risks and socioeconomic disadvantages are distributed across space. In large cities such as Chicago, environmental conditions and socioeconomic characteristics vary substantially across neighborhoods due to historical patterns of development, infrastructure investment, and industrial land use.

This project examines the spatial relationship between environmental exposure and socioeconomic vulnerability across Chicago census tracts. Rather than assuming a direct causal relationship between the two, the analysis treats environmental exposure and socioeconomic vulnerability as parallel dimensions of environmental inequality. By constructing tract-level indices and visualizing their spatial overlap, the project identifies neighborhoods where environmental risks and socioeconomic disadvantage coincide.

Data and Variables

The analysis combines environmental and socioeconomic data at the census tract level. Environmental indicators were derived from remote sensing and administrative datasets. Satellite imagery from Landsat 8, accessed through Google Earth Engine, was used to measure land surface temperature and vegetation conditions across Chicago. Additional environmental indicators were constructed from spatial datasets including road networks and the Environmental Protection Agency's Toxic Release Inventory (TRI). These datasets capture several dimensions of environmental exposure including urban heat, vegetation coverage, built environment intensity, transportation-related pollution, and industrial emissions.

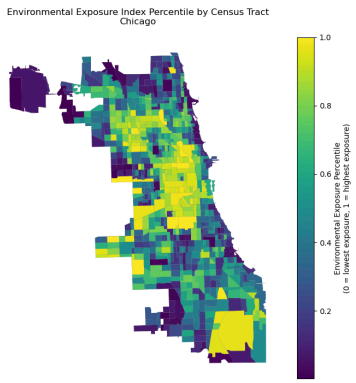
Socioeconomic indicators were obtained from the American Community Survey. The analysis includes median household income, renter share, housing age, and median home value. These indicators capture structural socioeconomic conditions that may influence communities' vulnerability to environmental stressors and their capacity to respond to environmental risks.

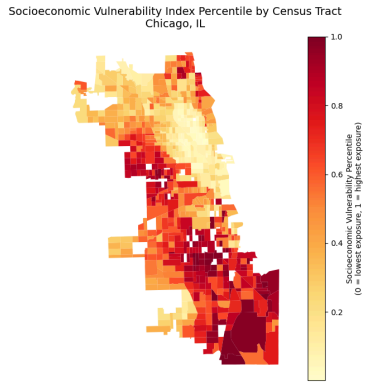
Index Construction

To make variables comparable across different units and scales, all indicators were standardized using z-scores. This transformation centers each variable around the citywide mean and expresses values relative to the overall distribution. Higher values therefore represent greater environmental exposure or greater socioeconomic vulnerability compared to the citywide average.

The environmental exposure index combines land surface temperature, vegetation coverage, impervious surface share, road density, and industrial emissions. The socioeconomic vulnerability index combines income, renter share, housing age, and home value. Weights were applied to reflect the relative importance of these indicators. Variables

associated with direct environmental burdens receive greater weight in the environmental index, while income and housing value receive greater weight in the socioeconomic index because they capture broader economic capacity and neighborhood resources.





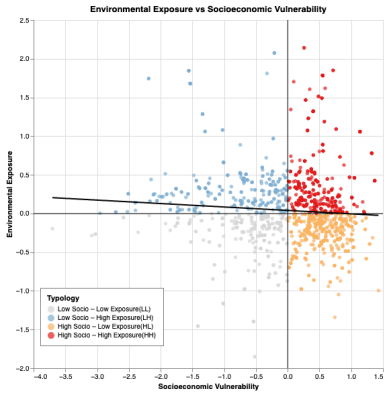


Figure 1. Environmental Exposure Index

Figure 2. Socioeconomic Vulnerability Index Percentile

Figure 3. Environmental Exposure vs. Socioeconomic Vulnerability (Scatterplot)

Typology and Scatterplot Analysis

To examine how environmental exposure and socioeconomic vulnerability intersect spatially, census tracts were classified using a two-dimensional typology. Each tract is positioned according to its environmental exposure score and socioeconomic vulnerability score. Based on whether each score falls above or below the citywide average, tracts are grouped into four categories representing different combinations of exposure and vulnerability.

Figure 3 presents a scatterplot of environmental exposure and socioeconomic vulnerability across Chicago census tracts. Each point represents a tract positioned according to its standardized index values. Reference lines divide the plot into four quadrants corresponding to the typology categories. This visualization highlights how tracts are distributed across the two dimensions and identifies areas where both exposure and vulnerability are relatively high.

Because the typology depends on threshold cutoffs, the classification of tracts is sensitive to the chosen threshold. If the threshold used to define high exposure or high vulnerability were raised, fewer tracts would fall into the high categories. Conversely, lowering the threshold would increase the number of tracts classified as experiencing high exposure or high vulnerability. The scatterplot therefore illustrates how the typology reflects relative comparisons within the city rather than fixed categories.

Spatial Patterns

Figures 1 and 2 present the static maps of the environmental exposure index and the socioeconomic vulnerability index across Chicago census tracts. The environmental exposure map reveals clear spatial variation across the city. Higher exposure scores appear in areas characterized by dense infrastructure, major transportation corridors, and proximity to industrial facilities, where elevated heat levels, limited vegetation, and pollution sources overlap.

The socioeconomic vulnerability map highlights neighborhoods with lower income levels, higher renter shares, and older housing stock. These patterns reflect long-standing socioeconomic disparities across the city.

When considered together, these maps show that environmental exposure and socioeconomic vulnerability are unevenly distributed across Chicago. Some neighborhoods experience both higher environmental burdens and greater social vulnerability. These areas correspond to tracts located in the high exposure–high vulnerability quadrant of the typology and represent potential environmental justice priority areas.

Interactive Visualization

To complement the static visualizations, an interactive dashboard was developed using Streamlit. The dashboard allows users to explore environmental exposure and socioeconomic vulnerability across Chicago census tracts through an interactive map and scatterplot.

The dashboard includes a dynamic scatterplot that allows users to examine how typology classifications change under different threshold definitions. Users can classify tracts using the mean of standardized scores, the median of the distribution, or a custom threshold selected by the user. Because the typology is defined by threshold cutoffs, adjusting the threshold mechanically shifts some tracts between categories. The interactive visualization therefore helps illustrate how classification outcomes depend on analytical choices.

The dashboard also includes a table showing the communities with the highest environmental exposure scores. This feature highlights areas experiencing the greatest environmental burdens and allows users to quickly identify neighborhoods that may require closer policy attention.

Limitations

Several limitations should be acknowledged. Environmental indicators rely partly on remote sensing data derived from Landsat imagery. Processing multi-year satellite data directly can be computationally intensive, so Google Earth Engine was used to efficiently access and process these datasets.

In addition, the typology classification depends on the threshold used to distinguish high and low values. Alternative threshold definitions, such as percentile-based cutoffs, could lead to different group memberships. Despite these limitations, the analysis provides a useful framework for examining how environmental exposure and socioeconomic vulnerability intersect spatially across Chicago.

Policy Implications

The combined analysis of environmental exposure and socioeconomic vulnerability provides a framework for identifying potential environmental justice priority areas in Chicago. Neighborhoods that fall into the high exposure–high vulnerability category may face both greater environmental burdens and fewer socioeconomic resources to respond to those risks. These areas may therefore warrant closer attention from policymakers when allocating environmental monitoring, urban greening initiatives, or infrastructure investments.

The results also illustrate how spatial data analysis can support more targeted environmental policy decisions. By identifying where environmental exposure and socioeconomic vulnerability overlap, policymakers can better prioritize interventions that reduce environmental risks while also addressing underlying socioeconomic inequalities.

Final Project

Cassie & Hani

0. Loading

```
import pandas as pd
from pathlib import Path
import os
import geopandas as gpd
from shapely.geometry import Point
import numpy as np

import matplotlib.pyplot as plt
import altair as alt

import rasterio
from rasterio.warp import calculate_default_transform, reproject, Resampling
from rasterio.mask import mask
from rasterstats import zonal_stats

from IPython.display import Image
```

For SocioEconomic Vulnerability Index

```
# 0. Preprocessing

DATA_DIR = Path("data/raw-data")

income = pd.read_csv(DATA_DIR / "income_24.csv")
renter = pd.read_csv(DATA_DIR / "renter_24.csv")
housing_age = pd.read_csv(DATA_DIR / "housing_age_24.csv")
home_value = pd.read_csv(DATA_DIR / "home_value_24.csv")
```

```python
### cleaning the geo id
def clean_geoid(df):
    possible_cols = ['GEO_ID', 'geo_id', 'index']

    for col in possible_cols:
        if col in df.columns:
            df['GEOID'] = df[col].astype(str)
            break

    # remove '1400000US'
    df['GEOID'] = df['GEOID'].str.replace('1400000US', '', regex=False)
    df['GEOID'] = df['GEOID'].str.zfill(11)

    return df

income = clean_geoid(income)
renter = clean_geoid(renter)
housing_age = clean_geoid(housing_age)
home_value = clean_geoid(home_value)

### income
income.head()
income = income.rename(columns = {'B19013_001E': 'median_income'})
income['median_income'].dtype
income['median_income'] = pd.to_numeric(income['median_income'],
 ↪  errors='coerce')

income = income.dropna(subset = 'median_income')
income = income[['GEOID', 'NAME', 'median_income']]

income['median_income'].describe()
income.head()

### renter share
renter.head()
renter = renter.rename(columns={
    'B25003_001E': 'total_units',
    'B25003_003E': 'renter_units'
})
print(renter[['total_units','renter_units']].dtypes)
renter['total_units'] = pd.to_numeric(renter['total_units'], errors='coerce')
renter['renter_units'] = pd.to_numeric(renter['renter_units'],
 ↪  errors='coerce')
```

```
print(renter[['total_units','renter_units']].dtypes)

renter = renter.dropna(subset=['total_units', 'renter_units'])
renter = renter[renter['total_units'] > 0]
renter['renter_share'] = renter['renter_units'] / renter['total_units']
renter = renter[['GEOID', 'NAME', 'total_units', 'renter_units',
 ↪  'renter_share']]

print(renter['renter_share'].describe())
renter.head()

### housing age
for col in housing_age.columns:
    if col not in ['GEOID', 'NAME'] and housing_age[col].dtype == 'object':
        housing_age[col] = pd.to_numeric(
            housing_age[col].astype(str)
                              .str.replace(',', '', regex=False)
                              .str.strip(),
            errors='coerce'
        )
housing_age.head()

housing_age = housing_age.rename(columns={
    'B25034_001E': 'total_units',
    'B25034_008E': 'built_1960_69',
    'B25034_009E': 'built_1950_59',
    'B25034_010E': 'built_1940_49',
    'B25034_011E': 'built_1939_before'
})

housing_age['old_units'] = (
    housing_age['built_1960_69'] +
    housing_age['built_1950_59'] +
    housing_age['built_1940_49'] +
    housing_age['built_1939_before']
)
housing_age = housing_age.dropna(subset = 'old_units')

housing_age = housing_age[housing_age['total_units'] > 0]
housing_age['old_housing_share'] = ((housing_age['old_units']
  / housing_age['total_units']))

housing_age = (housing_age[['GEOID', 'NAME', 'total_units',
```

```
  'old_units', 'old_housing_share']])
print(housing_age['old_housing_share'].describe())
housing_age.head()

### home value
home_value.head()
home_value = home_value.rename(columns = {'B25077_001E':
 ↪  'median_home_value'})

home_value['median_home_value'] =
 ↪  (pd.to_numeric(home_value['median_home_value'],
  errors='coerce'))
home_value = home_value.dropna(subset = 'median_home_value')

home_value = home_value[['GEOID', 'NAME', 'median_home_value']]
print(home_value['median_home_value'].describe())
home_value.head()

income.to_csv('income_clean.csv', index=False)
renter.to_csv('renter_clean.csv', index=False)
housing_age.to_csv('housing_age_clean.csv', index=False)
home_value.to_csv('home_value_clean.csv', index=False)
```

```
total_units     object
renter_units    object
dtype: object
total_units     float64
renter_units    float64
dtype: object
count    1328.000000
mean        0.425721
std         0.236789
min         0.000000
25%         0.228092
50%         0.426208
75%         0.606256
max         1.000000
Name: renter_share, dtype: float64
count    1328.000000
mean        0.647364
std         0.242973
min         0.005435
```

```
25%         0.496404
50%         0.713576
75%         0.838772
max         1.000000
Name: old_housing_share, dtype: float64
count    1.305000e+03
mean     3.574545e+05
std      1.940016e+05
min      3.170000e+04
25%      2.300000e+05
50%      3.147000e+05
75%      4.268000e+05
max      1.673200e+06
Name: median_home_value, dtype: float64
```

1. Reading in all the file

```
DATA_CLEAN_DIR = Path("data/derived-data")
DATA_DIR = Path("data/raw-data")

income = pd.read_csv(DATA_CLEAN_DIR / "income_clean.csv")
renter = pd.read_csv(DATA_CLEAN_DIR / "renter_clean.csv")
housing_age = pd.read_csv(DATA_CLEAN_DIR / "housing_age_clean.csv")
home_value = pd.read_csv(DATA_CLEAN_DIR / "home_value_clean.csv")

tracts = gpd.read_file(DATA_DIR / 'tl_2023_17_tract'/ "tl_2023_17_tract.shp")
```

2. Clip All Dataframe into Chicago Shape

```
tracts = gpd.read_file(DATA_DIR / 'tl_2023_17_tract'/ "tl_2023_17_tract.shp")
tracts["GEOID"] = tracts["GEOID"].astype(str)
cook_tracts = tracts[tracts["COUNTYFP"] == "031"].copy()

def make_it_geo(df, base_map):
    df["GEOID"] = df["GEOID"].astype(str)
    return base_map.merge(df, on="GEOID", how="inner")

gdf_income = make_it_geo(income, cook_tracts)
gdf_renter = make_it_geo(renter, cook_tracts)
gdf_age = make_it_geo(housing_age, cook_tracts)
gdf_home = make_it_geo(home_value, cook_tracts)
```

```python
city = gpd.read_file(DATA_DIR / "Boundaries_Chicago.geojson")
city = city.to_crs(gdf_income.crs)
city_union = city.geometry.unary_union

def clip_to_chicago_and_save(gdf, name):
    gdf = gdf.to_crs(city.crs)

    gdf_chi = gdf[gdf.geometry.centroid.within(city_union)].copy()

    # as csv
    cols_to_save = [c for c in gdf_chi.columns if c != 'geometry']
    csv_filename = f"{name}_chicago.csv"
    gdf_chi[cols_to_save].to_csv(csv_filename, index=False)

    # as gpkg
    gdf_chi.crs = "EPSG:4326"
    gpkg_filename = f"{name}_chicago.gpkg"
    gdf_chi.to_file(gpkg_filename, layer='tracts', driver="GPKG")

    print(f" {name} done:")
    print(f"   - Number of Tracts being kept: {len(gdf_chi)}")
    print(f"   - Saved: {csv_filename} & {gpkg_filename}")
    return gdf_chi

gdfs = ({"income_24c": gdf_income, "renter_24c": gdf_renter,
  "house_age_24c": gdf_age, "home_value_24c": gdf_home})
chicago_gdfs = {}

for name, target_gdf in gdfs.items():
    chicago_gdfs[name] = clip_to_chicago_and_save(target_gdf, name)

income_chi = chicago_gdfs["income_24c"]
renter_chi = chicago_gdfs["renter_24c"]
age_chi = chicago_gdfs["house_age_24c"]
home_value_chi = chicago_gdfs["home_value_24c"]
```

```
/var/folders/wk/6jzgbh0j3p5gxs8h2w19px0m0000gn/T/ipykernel_8853/3841772996.py:16: Deprecatio
  city_union = city.geometry.unary_union
/var/folders/wk/6jzgbh0j3p5gxs8h2w19px0m0000gn/T/ipykernel_8853/3841772996.py:21: UserWarning

  gdf_chi = gdf[gdf.geometry.centroid.within(city_union)].copy()
```

```
 income_24c done:
   - Number of Tracts being kept: 761
   - Saved: income_24c_chicago.csv & income_24c_chicago.gpkg

/var/folders/wk/6jzgbh0j3p5gxs8h2w19px0m0000gn/T/ipykernel_8853/3841772996.py:21: UserWarning

  gdf_chi = gdf[gdf.geometry.centroid.within(city_union)].copy()

 renter_24c done:
   - Number of Tracts being kept: 778
   - Saved: renter_24c_chicago.csv & renter_24c_chicago.gpkg

/var/folders/wk/6jzgbh0j3p5gxs8h2w19px0m0000gn/T/ipykernel_8853/3841772996.py:21: UserWarning

  gdf_chi = gdf[gdf.geometry.centroid.within(city_union)].copy()

 house_age_24c done:
   - Number of Tracts being kept: 778
   - Saved: house_age_24c_chicago.csv & house_age_24c_chicago.gpkg

/var/folders/wk/6jzgbh0j3p5gxs8h2w19px0m0000gn/T/ipykernel_8853/3841772996.py:21: UserWarning

  gdf_chi = gdf[gdf.geometry.centroid.within(city_union)].copy()

 home_value_24c done:
   - Number of Tracts being kept: 759
   - Saved: home_value_24c_chicago.csv & home_value_24c_chicago.gpkg
```

3. Standardizing all variables

```python
def zscore(series):
    return (series - series.mean()) / series.std()

income_chi['income_z'] = -zscore(income_chi['median_income'])
renter_chi['renter_z'] = zscore(renter_chi['renter_share'])
age_chi['age_z'] = zscore(age_chi['old_housing_share'])
home_value_chi['home_value_z'] = -zscore(home_value_chi['median_home_value'])
```

4. Sanity Check for each variable

```python
print(income_chi['income_z'].describe())
print(renter_chi['renter_z'].describe())
print(age_chi['age_z'].describe())
print(home_value_chi['home_value_z'].describe())
```

```
count    7.610000e+02
mean     2.801088e-17
std      1.000000e+00
min     -4.131157e+00
25%     -5.060247e-01
50%      1.811047e-01
75%      7.163770e-01
max      1.630230e+00
Name: income_z, dtype: float64
count    7.780000e+02
mean    -1.187282e-16
std      1.000000e+00
min     -2.405459e+00
25%     -7.076801e-01
50%      6.936867e-02
75%      7.284943e-01
max      2.246324e+00
Name: renter_z, dtype: float64
count    7.780000e+02
mean     4.566470e-18
std      1.000000e+00
min     -3.533125e+00
25%     -4.308358e-01
50%      2.211402e-01
75%      7.510398e-01
max      1.463366e+00
Name: age_z, dtype: float64
count    7.590000e+02
mean     4.680782e-17
std      1.000000e+00
min     -6.310022e+00
25%     -4.306332e-01
50%      2.027926e-01
75%      7.172688e-01
max      1.521039e+00
Name: home_value_z, dtype: float64
```

5. Setting up the Socio Index

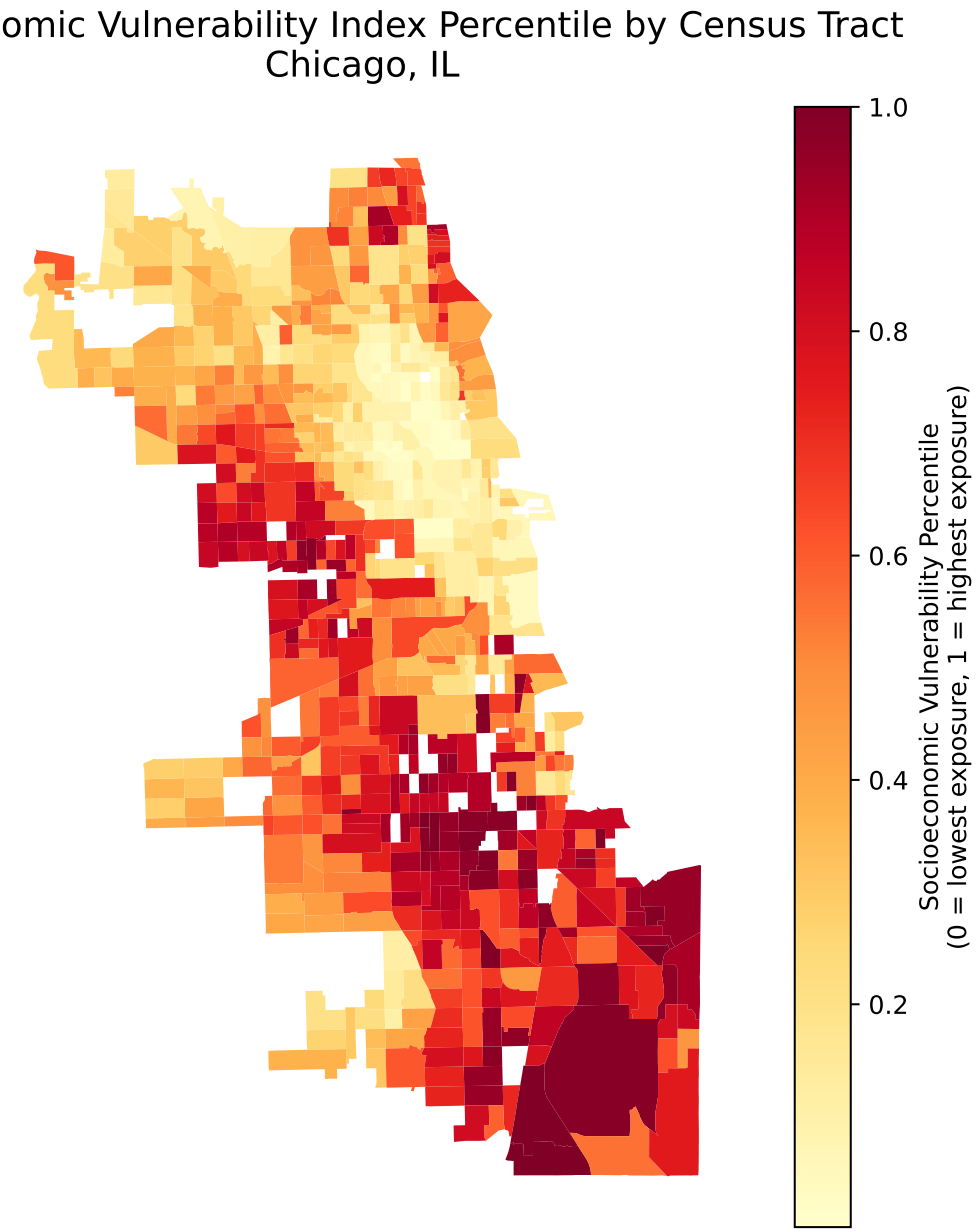
```
socio = income_chi[['GEOID', 'geometry', 'income_z']].merge(
    renter_chi[['GEOID', 'renter_z']],
    on='GEOID', how = 'inner').merge(
    age_chi[['GEOID', 'age_z']],
    on='GEOID', how = 'inner').merge(
    home_value_chi[['GEOID', 'home_value_z']],
    on='GEOID', how = 'inner')

socio['socio_index'] = (
    2 * socio['income_z'] +
    0.5 * socio['renter_z'] +
    0.5 * socio['age_z'] +
    2 * socio['home_value_z']) / 5

socio['socio_index'].describe()
```

```
count    742.000000
mean      -0.018065
std        0.807717
min       -3.703724
25%       -0.417484
50%        0.179886
75%        0.558521
max        1.428754
Name: socio_index, dtype: float64
```

6. Mapping socio vulnerability index

```
socio["socio_percentile"] = (
    socio["socio_index"]
    .rank(pct=True)
)

max_abs = abs(socio['socio_index']).max()
fig, ax = plt.subplots(figsize=(7,7))

socio.plot(
    column='socio_percentile',
    cmap='YlOrRd',
    linewidth = 0.1,
```

```
    legend=True,
    legend_kwds=({"label": "Socioeconomic Vulnerability Percentile"
      "\n(0 = lowest exposure, 1 = highest exposure)","orientation":
       ↪  "vertical"}),
    ax=ax
)

ax.set_title("Socioeconomic Vulnerability Index Percentile by Census Tract"
    "\nChicago, IL",
    fontsize=14,
    pad=12)

ax.set_axis_off()

plt.tight_layout()
plt.show()
```



For Environmental Exposure Index

1. Pre-processing (CRS + Clipping)

```python
ENV_RAW_DIR = Path("data/raw-data")
ENV_DERIVED_DIR = Path("data/derived-data")
ENV_DERIVED_DIR.mkdir(parents=True, exist_ok=True)

TARGET_CRS = "EPSG:3435"

TRACTS_PATH  = ENV_RAW_DIR / "tl_2022_17_tract.shp"
CHICAGO_PATH = ENV_RAW_DIR / "Boundaries_Chicago.geojson"
ROADS_PATH   = ENV_RAW_DIR / "tl_2022_17_prisecroads.shp"
TRI_CSV_PATH = ENV_RAW_DIR / "EPA_TRI_IL_2022.csv"

RASTERS = {
    "lst_summer":  ENV_RAW_DIR / "chicago_summer_LST.tif",
    "lst_winter":  ENV_RAW_DIR / "chicago_winter_LST.tif",
    "ndvi_summer": ENV_RAW_DIR / "chicago_summer_NDVI.tif",
    "impervious":  ENV_RAW_DIR / "chicago_impervious.tif",
}

OUT_TRACTS_GPKG = ENV_DERIVED_DIR / "env__tracts_chicago_2022.gpkg"
OUT_ROADS_GPKG  = ENV_DERIVED_DIR / "env__roads_chicago_2022.gpkg"
OUT_TRI_GPKG    = ENV_DERIVED_DIR / "env__tri_chicago_2022.gpkg"

def _drop_empty(gdf: gpd.GeoDataFrame) -> gpd.GeoDataFrame:
    return gdf[gdf.geometry.notna() & ~gdf.geometry.is_empty].copy()

def _fix_geom(gdf: gpd.GeoDataFrame) -> gpd.GeoDataFrame:
    gdf = gdf.copy()
    gdf["geometry"] = gdf.geometry.make_valid()
    return gdf

def _dissolve_to_single_polygon(gdf: gpd.GeoDataFrame) -> gpd.GeoDataFrame:
    return gdf.dissolve()

def _reproject_and_clip_raster_to_mask(
    src_path: Path,
    dst_path: Path,
    mask_gdf: gpd.GeoDataFrame,
    target_crs: str = TARGET_CRS
) -> None:
    """
    Reproject raster to target CRS, then clip to mask geometry.
    Writes GeoTIFF to dst_path.
    """
```

```python
    with rasterio.open(src_path) as src:
        if src.crs is None:
            raise ValueError(f"Raster has no CRS: {src_path}")

        transform, width, height = calculate_default_transform(
            src.crs, target_crs, src.width, src.height, *src.bounds
        )

        dst_meta = src.meta.copy()
        dst_meta.update({
            "crs": target_crs,
            "transform": transform,
            "width": width,
            "height": height
        })

        tmp_path = ENV_DERIVED_DIR / f"__tmp_reproject_{src_path.stem}.tif"
        with rasterio.open(tmp_path, "w", **dst_meta) as dst:
            for i in range(1, src.count + 1):
                reproject(
                    source=rasterio.band(src, i),
                    destination=rasterio.band(dst, i),
                    src_transform=src.transform,
                    src_crs=src.crs,
                    dst_transform=transform,
                    dst_crs=target_crs,
                    resampling=Resampling.bilinear,
                )

    with rasterio.open(tmp_path) as reproj:
        geoms = list(mask_gdf.geometry)
        out_img, out_transform = mask(reproj, geoms, crop=True)
        out_meta = reproj.meta.copy()
        out_meta.update({
            "height": out_img.shape[1],
            "width": out_img.shape[2],
            "transform": out_transform
        })

    with rasterio.open(dst_path, "w", **out_meta) as out:
        out.write(out_img)

    try:
```

```
        tmp_path.unlink()
    except OSError:
        pass

def _add_zonal_mean(
    gdf: gpd.GeoDataFrame,
    raster_path: Path,
    out_col: str) -> gpd.GeoDataFrame:
    zs = zonal_stats(gdf, raster_path, stats=["mean"], all_touched=False)
    gdf[out_col] = [d["mean"] for d in zs]
    return gdf


env_tracts = _fix_geom(_drop_empty(gpd.read_file(TRACTS_PATH)))
env_chicago = _fix_geom(_drop_empty(gpd.read_file(CHICAGO_PATH)))

env_tracts = env_tracts.to_crs(TARGET_CRS)
env_chicago = env_chicago.to_crs(TARGET_CRS)

env_chicago_poly = _dissolve_to_single_polygon(env_chicago)
env_tracts_chicago = gpd.clip(env_tracts, env_chicago_poly)
print("Chicago tracts:", len(env_tracts_chicago))

env_roads =
 ↪  _fix_geom(_drop_empty(gpd.read_file(ROADS_PATH))).to_crs(TARGET_CRS)
env_roads_chicago = gpd.clip(env_roads, env_chicago_poly)
print("Road segments (clipped):", len(env_roads_chicago))

env_tri = pd.read_csv(TRI_CSV_PATH)
env_tri = env_tri.dropna(subset=["12. LATITUDE", "13. LONGITUDE"]).copy()

env_tri_gdf = gpd.GeoDataFrame(
    env_tri,
    geometry=gpd.points_from_xy(env_tri["13. LONGITUDE"], env_tri["12.
 ↪  LATITUDE"]),
    crs="EPSG:4326"
).to_crs(TARGET_CRS)

env_tri_gdf = gpd.clip(env_tri_gdf, env_chicago_poly)
print("TRI points (clipped):", len(env_tri_gdf))

env_tracts_chicago.to_file(OUT_TRACTS_GPKG, layer="tracts", driver="GPKG")
env_roads_chicago.to_file(OUT_ROADS_GPKG, layer="roads", driver="GPKG")
```

```
env_tri_gdf.to_file(OUT_TRI_GPKG, layer="tri", driver="GPKG")


for name, path in RASTERS.items():
    out_path = ENV_DERIVED_DIR / f"env__{name}_3435_clip.tif"
    _reproject_and_clip_raster_to_mask(path, out_path, env_chicago_poly,
 ↪  target_crs=TARGET_CRS)

print("Rasters reprojected & clipped to", TARGET_CRS)


env_tracts = gpd.read_file(OUT_TRACTS_GPKG, layer="tracts")
env_roads  = gpd.read_file(OUT_ROADS_GPKG, layer="roads")
env_tri    = gpd.read_file(OUT_TRI_GPKG, layer="tri")

env_tracts = _add_zonal_mean(env_tracts, ENV_DERIVED_DIR /
 ↪  "env__lst_summer_3435_clip.tif", "lst_summer_mean")
env_tracts = _add_zonal_mean(env_tracts, ENV_DERIVED_DIR /
 ↪  "env__lst_winter_3435_clip.tif", "lst_winter_mean")
env_tracts = _add_zonal_mean(env_tracts, ENV_DERIVED_DIR /
 ↪  "env__ndvi_summer_3435_clip.tif", "ndvi_mean")
env_tracts = _add_zonal_mean(env_tracts, ENV_DERIVED_DIR /
 ↪  "env__impervious_3435_clip.tif", "impervious_mean")

env_roads_x = gpd.overlay(
    env_roads[["geometry"]],
    env_tracts[["GEOID", "geometry"]],
    how="intersection"
)
env_roads_x["road_len_ft"] = env_roads_x.geometry.length

env_road_sum =
 ↪  env_roads_x.groupby("GEOID")["road_len_ft"].sum().reset_index()

env_tracts["tract_area_ft2"] = env_tracts.geometry.area
env_tracts = env_tracts.merge(env_road_sum, on="GEOID", how="left")
env_tracts["road_len_ft"] = env_tracts["road_len_ft"].fillna(0)
env_tracts["road_density"] = env_tracts["road_len_ft"] /
 ↪  env_tracts["tract_area_ft2"]

env_tri = env_tri.copy()
env_tri["total_releases"] = pd.to_numeric(env_tri["107. TOTAL RELEASES"],
 ↪  errors="coerce").fillna(0)
```

```python
env_tri_join = gpd.sjoin(
    env_tri,
    env_tracts[["GEOID", "geometry"]],
    how="inner",
    predicate="within"
)

env_tri_summary = (
    env_tri_join.groupby("GEOID")
    .agg(
        tri_release_total=("total_releases", "sum"),
        tri_facility_count=("2. TRIFD", "nunique")
    )
    .reset_index()
)

env_tracts = env_tracts.merge(env_tri_summary, on="GEOID", how="left")
env_tracts["tri_release_total"] = env_tracts["tri_release_total"].fillna(0)
env_tracts["tri_facility_count"] = env_tracts["tri_facility_count"].fillna(0)

env_tracts.to_file(ENV_DERIVED_DIR / "env__metrics_tracts.gpkg",
 ↪  layer="tracts", driver="GPKG")
env_tracts.drop(columns="geometry").to_csv(ENV_DERIVED_DIR /
 ↪  "env__metrics_tracts.csv", index=False)

print("\n  Done. Environmental metrics saved to:")
print(" -", (ENV_DERIVED_DIR / "env__metrics_tracts.gpkg"))
print(" -", (ENV_DERIVED_DIR / "env__metrics_tracts.csv"))
```

```
/opt/miniconda3/envs/DAP/lib/python3.12/site-packages/shapely/set_operations.py:168: RuntimeW
  return lib.intersection(a, b, **kwargs)
/opt/miniconda3/envs/DAP/lib/python3.12/site-packages/shapely/set_operations.py:168: RuntimeW
  return lib.intersection(a, b, **kwargs)

Chicago tracts: 862
Road segments (clipped): 276
TRI points (clipped): 249
Rasters reprojected & clipped to EPSG:3435

/opt/miniconda3/envs/DAP/lib/python3.12/site-packages/rasterstats/io.py:335: NodataWarning: S
  warnings.warn(
```

```
 Done. Environmental metrics saved to:
 - data/derived-data/env__metrics_tracts.gpkg
 - data/derived-data/env__metrics_tracts.csv


/opt/miniconda3/envs/DAP/lib/python3.12/site-packages/shapely/set_operations.py:168: RuntimeW
  return lib.intersection(a, b, **kwargs)
/var/folders/wk/6jzgbh0j3p5gxs8h2w19px0m0000gn/T/ipykernel_8853/540311296.py:147: UserWarning
  env_roads_x = gpd.overlay(
```

2. Load tract-level environmental metrics

```python
DERIVED_DIR = Path("data/derived-data")
RAW_DIR = Path("data/raw-data")

env = gpd.read_file(DERIVED_DIR / "env__metrics_tracts.gpkg", layer="tracts")

expected_cols = [
    "GEOID",
    "lst_summer_mean",
    "lst_winter_mean",
    "ndvi_mean",
    "impervious_mean",
    "road_density",
    "tri_release_total",
    "tri_facility_count",
]
missing = [c for c in expected_cols if c not in env.columns]

print("\nRows:", len(env))
print("\nBasic summary (selected columns):")
print(env[["lst_summer_mean","ndvi_mean","impervious_mean","road_density",
  "tri_release_total"]].describe())
```

```
Rows: 862

Basic summary (selected columns):
       lst_summer_mean   ndvi_mean  impervious_mean  road_density  \
count       831.000000  831.000000       825.000000    862.000000
mean         31.786221    0.309573        71.249681      0.034706
std           2.129325    0.089562        12.534633      0.296620
```

```
min          17.987406   -0.014050         0.000000      0.000000
25%          30.732338    0.247801        65.602626      0.000000
50%          32.002551    0.300752        72.916435      0.000000
75%          33.218334    0.367939        79.374214      0.000755
max          36.756376    0.668505        95.000000      3.886548

       tri_release_total
count         862.000000
mean         4125.077160
std         48414.604146
min             0.000000
25%             0.000000
50%             0.000000
75%             0.000000
max        856756.000000
```

3. Cleaning + transformations

```
env = env.dropna(subset=["lst_summer_mean", "ndvi_mean",
↪  "impervious_mean"]).copy()
env["ndvi_bad"] = -env["ndvi_mean"]
env["tri_release_log"] = np.log1p(env["tri_release_total"])

print("\nAfter cleaning, rows:", len(env))
```

```
After cleaning, rows: 819
```

4. Z-score standardization + weighted exposure index

```
vars_for_index = [
    "lst_summer_mean",
    "ndvi_bad",
    "impervious_mean",
    "road_density",
    "tri_release_log",
]

for v in vars_for_index:
    env[f"z_{v}"] = (env[v] - env[v].mean()) / env[v].std()

weights = {
```

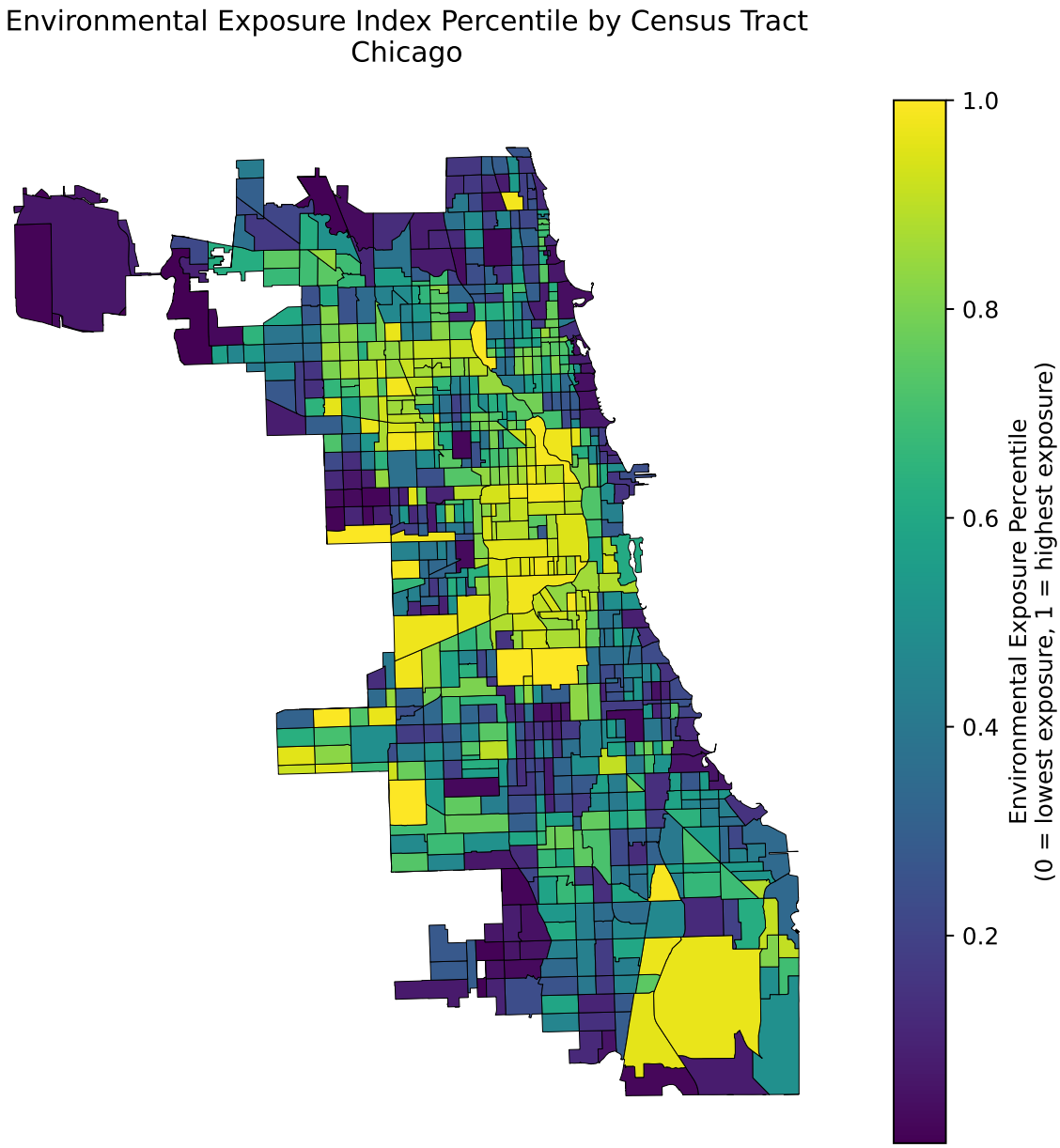
```
    "z_lst_summer_mean": 0.25,
    "z_ndvi_bad": 0.125,
    "z_impervious_mean": 0.125,
    "z_road_density": 0.25,
    "z_tri_release_log": 0.25,
}

env["env_exposure_index"] = sum(env[k] * w for k, w in weights.items())

env["exposure_percentile"] = env["env_exposure_index"].rank(pct=True)

print("\nExposure index summary:")
print(env["env_exposure_index"].describe())
```

```
Exposure index summary:
count    8.190000e+02
mean     2.353293e-16
std      5.427220e-01
min     -2.097397e+00
25%     -2.316378e-01
50%     -3.387435e-02
75%      1.926787e-01
max      7.937837e+00
Name: env_exposure_index, dtype: float64
```

5. Visualization

```
fig, ax = plt.subplots(figsize=(7,7))

env.plot(
    column="exposure_percentile",
    cmap="viridis",
    legend=True,
    linewidth=0.05,
    edgecolor="black",
    ax=ax,
    legend_kwds={
        "label": "Environmental Exposure Percentile\n(0 = lowest exposure, 1
        ↪  = highest exposure)",
        "orientation": "vertical"
    }
```

```
)

ax.set_title("Environmental Exposure Index Percentile by Census
↪  Tract\nChicago")
ax.set_axis_off()

plt.tight_layout()
plt.show()
```



6. Final outputs

```
env.to_file(DERIVED_DIR / "env__exposure_final.gpkg", layer="tracts",
↪  driver="GPKG")
env.drop(columns="geometry").to_csv(DERIVED_DIR / "env__exposure_final.csv",
↪  index=False)
```

```
print("\n  Saved final outputs:")
print(" -", DERIVED_DIR / "env__exposure_final.gpkg")
print(" -", DERIVED_DIR / "env__exposure_final.csv")
```

```
  Saved final outputs:
 - data/derived-data/env__exposure_final.gpkg
 - data/derived-data/env__exposure_final.csv
```

Combine 2 index & Visualization

1. Combining two indexes

```
socio.columns
tracts.columns

socio['GEOID'] = socio['GEOID'].astype(str)
env['GEOID'] = env['GEOID'].astype(str)

combined = env.merge(
    socio[['GEOID','socio_index']],
    on='GEOID',
    how='inner'
)
combined.columns
print(combined.head())
```

```
  STATEFP COUNTYFP TRACTCE        GEOID     NAME              NAMELSAD  MTFCC  \
0      17      031  740400  17031740400     7404     Census Tract 7404  G5020
1      17      031  530503  17031530503  5305.03  Census Tract 5305.03  G5020
2      17      031  530502  17031530502  5305.02  Census Tract 5305.02  G5020
3      17      031  530600  17031530600     5306     Census Tract 5306  G5020
4      17      031  530400  17031530400     5304     Census Tract 5304  G5020

  FUNCSTAT    ALAND  AWATER  ...  ndvi_bad tri_release_log  z_lst_summer_mean  \
0        S  2838804       0  ... -0.483181             0.0           0.012139
1        S  1598487   57148  ... -0.465101             0.0           0.041206
2        S   706252  101712  ... -0.515984             0.0          -0.373541
3        S   966243   67959  ... -0.433670             0.0           0.819719
```

```
4        S  1312063       0  ... -0.407414             0.0           0.735237

   z_ndvi_bad  z_impervious_mean  z_road_density  z_tri_release_log  \
0   -1.954576          -2.515169       -0.043717          -0.214582
1   -1.749505          -1.436962       -0.040985          -0.214582
2   -2.326637          -1.760648       -0.044197          -0.214582
3   -1.393013          -1.237802       -0.044197          -0.214582
4   -1.095206          -0.403416       -0.040427          -0.214582

   env_exposure_index  exposure_percentile  socio_index
0           -0.620258             0.068376    -0.109405
1           -0.451899             0.114774     0.689428
2           -0.668991             0.054945     0.329627
3           -0.188617             0.290598     0.515577
4           -0.067271             0.456654     0.530705

[5 rows x 32 columns]
```

2. Add in community name in combined files

```
community = gpd.read_file(DATA_DIR /
 ↪  "Boundaries_-_Community_Areas_20260228.geojson")

# match CRS with tracts (important)
community = community.to_crs(combined.crs)

# remove any leftover index_right column (prevents join error)
if "index_right" in combined.columns:
    combined = combined.drop(columns="index_right")

# spatial join
combined = gpd.sjoin(combined,
    community[["community", "geometry"]],
    how="left",
    predicate="intersects"
)

# quick check
print(combined.head())
combined.columns

combined.to_file( DATA_CLEAN_DIR / "combined.geojson", driver="GeoJSON")
```

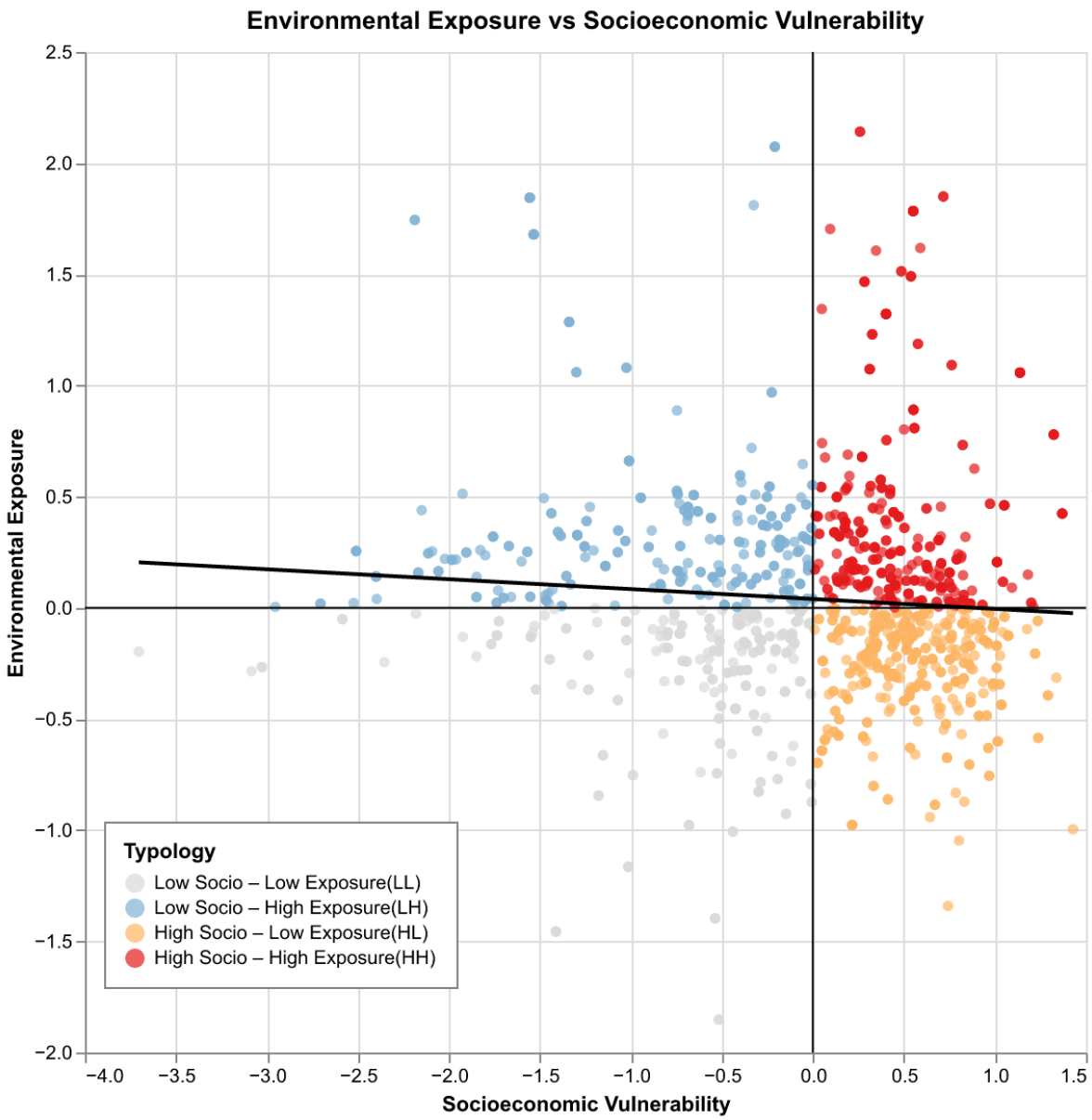
```
  STATEFP COUNTYFP TRACTCE        GEOID     NAME              NAMELSAD  MTFCC  \
0      17      031  740400  17031740400     7404     Census Tract 7404  G5020
1      17      031  530503  17031530503  5305.03  Census Tract 5305.03  G5020
2      17      031  530502  17031530502  5305.02  Census Tract 5305.02  G5020
3      17      031  530600  17031530600     5306     Census Tract 5306  G5020
3      17      031  530600  17031530600     5306     Census Tract 5306  G5020

  FUNCSTAT    ALAND  AWATER  ... z_lst_summer_mean z_ndvi_bad  \
0        S  2838804       0  ...          0.012139  -1.954576
1        S  1598487   57148  ...          0.041206  -1.749505
2        S   706252  101712  ...         -0.373541  -2.326637
3        S   966243   67959  ...          0.819719  -1.393013
3        S   966243   67959  ...          0.819719  -1.393013

   z_impervious_mean  z_road_density  z_tri_release_log  env_exposure_index  \
0          -2.515169       -0.043717          -0.214582           -0.620258
1          -1.436962       -0.040985          -0.214582           -0.451899
2          -1.760648       -0.044197          -0.214582           -0.668991
3          -1.237802       -0.044197          -0.214582           -0.188617
3          -1.237802       -0.044197          -0.214582           -0.188617

   exposure_percentile  socio_index  index_right        community
0             0.068376    -0.109405           73  MOUNT GREENWOOD
1             0.114774     0.689428           52     WEST PULLMAN
2             0.054945     0.329627           52     WEST PULLMAN
3             0.290598     0.515577           53        RIVERDALE
3             0.290598     0.515577           52     WEST PULLMAN

[5 rows x 34 columns]
```

3. Visualization for double indexes scatter plot

```
#quadrant + label + palette
combined["quadrant"] = (
    (combined["socio_index"] > 0).astype(int) * 2 +
    (combined["env_exposure_index"] > 0).astype(int)
)

quadrant_labels = {
    0: "Low Socio - Low Exposure(LL)",
    1: "Low Socio - High Exposure(LH)",
    2: "High Socio - Low Exposure(HL)",
    3: "High Socio - High Exposure(HH)"
```

```
}
combined["quadrant_label"] = combined["quadrant"].map(quadrant_labels)

palette = {
    "Low Socio - Low Exposure(LL)": "#d9d9d9",
    "Low Socio - High Exposure(LH)": "#80b1d3",
    "High Socio - Low Exposure(HL)": "#fdb462",
    "High Socio - High Exposure(HH)": "#e41a1c"
}

#Altair scatter + regression + reference lines
base = alt.Chart(combined).properties(
    width=520,
    height=520,
    title="Environmental Exposure vs Socioeconomic Vulnerability"
)

points = base.mark_circle(opacity=0.7, size=30).encode(
    x=alt.X("socio_index:Q", title="Socioeconomic Vulnerability"),
    y=alt.Y("env_exposure_index:Q", title="Environmental Exposure"),
    color=alt.Color(
        "quadrant_label:N",
        title="Typology",
        scale=alt.Scale(domain=list(palette.keys()),
 ↪  range=list(palette.values())),
        legend=alt.Legend(
            orient='none',
            legendX=10,
            legendY=400,
            fillColor='white',
            padding=10,
            strokeColor='gray')
    ),
    tooltip=[
        alt.Tooltip("quadrant_label:N", title="Typology"),
        alt.Tooltip("socio_index:Q", title="Socio", format=".3f"),
        alt.Tooltip("env_exposure_index:Q", title="Exposure", format=".3f"),
        alt.Tooltip("community:N", title="Community")
    ]
)

# regression line
reg_line = base.transform_regression(
```

```
    "socio_index", "env_exposure_index"
).mark_line(color="black").encode(
    x="socio_index:Q",
    y="env_exposure_index:Q"
)

# reference lines at x=0 and y=0
vline = alt.Chart(pd.DataFrame({"x":
 ↪  [0]})).mark_rule(color="black").encode(x="x:Q")
hline = alt.Chart(pd.DataFrame({"y":
 ↪  [0]})).mark_rule(color="black").encode(y="y:Q")

chart = (points + reg_line + vline + hline)

chart

chart.save('scatter.png', scale_factor = 2)
Image('scatter.png')
```


Environmental Exposure vs Socioeconomic Vulnerability
Environmental Exposure
Socioeconomic Vulnerability
2.5
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0
−4.0
−3.5
−3.0
−2.5
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
Typology
Low Socio – Low Exposure(LL)
Low Socio – High Exposure(LH)
High Socio – Low Exposure(HL)
High Socio – High Exposure(HH)